

Compulsory Assignment 3: Semantic Segmentation With U-net

Please fill out the the group name, number, members and optionally the name below.

Group number: 26

Member 1: Milad Tootoonchi

Member 2: Makka Dulgaeva

Member 3: Sania Minaipour

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we recommend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to segment the images using a U-Net. High accuracy is not required, as the purpose of this assignment is to evaluate the correct implementation of the U-Net architecture and workflow.

NOTE: It is recommended to use Google Colab with GPU enabled for CA3 task. Keras and TensorFlow (version ≥ 2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA3

To work on the CA3 assignment in Google Colab, follow these steps:

1. Connect Google Colab to Google Drive

Your CA3 files should be stored in Google Drive so Colab can access them.
Run the following code to mount your Drive:

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

After running this, you will be asked to grant access. Once mounted, your Drive will be available under `/content/drive/MyDrive/`.

2. Upload the CA3 Folder

- First, unzip the **CA3** folder on your local computer.
- Then upload the unzipped CA3 folder into your Google Drive (for example inside `MyDrive`).

Example path: `/content/drive/MyDrive/CA3`

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime** → **Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

3. Add CA3 to Python's Search Path

To make sure Python finds the `utilities.py` and other scripts in CA3, add the path (the path is just an example):

```
import sys
```

```
sys.path.append("/content/drive/MyDrive/CA3")
```

Adjust the path above to match the actual location of your CA3 folder in Google Drive.

4. Open the CA3 Notebook

- Upload the CA3 Jupyter notebook (for example, `CA3.ipynb`) to Colab.
 - Once opened, you can run the notebook and it will access the files in your CA3 folder.
-

5. Watch the First Lecture

The first lecture explains how to use Google Colab effectively.
Review it if you are new to Colab.

That's it — you are now ready to run your CA3 code on Google Colab.

Introduction

Chest X-ray(CXR) images are important in medical imaging because it helps focus analysis on the lung region, making disease detection (eg: pneumonia, COVID-19) more accurate and reliable. The purpose is to create clear lung masks that isolate the lungs from surrounding anatomy.

Lung segmentation is the process of separating lung regions from the rest of a chest X-ray. In this assignment, you are provided with CXR images and corresponding binary masks, where pixel values of 0 represent non-lung areas and 1 represents lung tissue. Your task is to focus on semantic segmentation using a U-Net-style architecture. You will segment lung regions from chest X-ray images.

Dataset

The dataset consists of 200 images with corresponding binary masks:

- **Images:** 200 chest X-ray images (grayscale).
- **Masks:** 200 binary masks.
 - 1 = lung
 - 0 = background

Structure

- Images: CXR_png/
- Masks: masks/

Mask filenames may use suffixes _mask .

Pairing rule: For X.png , expect X_mask.png (accepted suffixes).
You must programmatically verify pairs and report any missing masks.

Assignment Tasks

1. Build a file-pairing script to match each image with its mask; print a summary of matched pairs and missing files.
2. Create train/val/test splits (80/10/10) with a fixed random seed.

3. Implement preprocessing and mask-safe augmentations (flips, crop/zoom; nearest-neighbor resize for masks).
4. Implement the classical U-Net structure that you have learned about in the lectures.
5. Implement a U-Net with Transfer Learning (e.g., ResNet50, VGG16, MobileNetV2).
6. Train your models with a composite loss (e.g., $0.5 \cdot \text{BCE} + 0.5 \cdot \text{DiceLoss}$), track Dice and IoU metrics, and use callbacks (ModelCheckpoint, EarlyStopping).
7. Evaluate on the test set; report metrics and show qualitative results.

Task 0.1 : Data Organization & Pairing

- Build a file-pairing script to match each image with its mask.
- Print a summary of matched pairs and missing files.
- Show the distribution of training, validation and test set

Set DATA_ROOT to the path of the Images folder located in your CA3 assignment folder.

```
In [ ]: import os, glob, random
import numpy as np
import matplotlib.pyplot as plt
import warnings
import matplotlib
import tensorflow as tf

import importlib

import sys
sys.path.append('/content/drive/MyDrive/Colab Notebooks/CA3')

import utilities as ut
importlib.reload(ut)

DATA_ROOT = r"/content/drive/MyDrive/Colab Notebooks/CA3/Images"
IMG_DIR = os.path.join(DATA_ROOT, "CXR_png")
MSK_DIR = os.path.join(DATA_ROOT, "masks")

pairs, missing = ut.build_pairs(IMG_DIR, MSK_DIR)
ut.report_pairs(pairs, missing)
ut.peak_pairs(pairs, n_show=3)

train, val, test = ut.split_dataset(pairs, seed=42)
ut.plot_split_distribution(train, val, test)
```

Total images: 64
 Matched pairs: 64
 Missing masks: 0

Image CHNCXR_0059_0.png



Mask

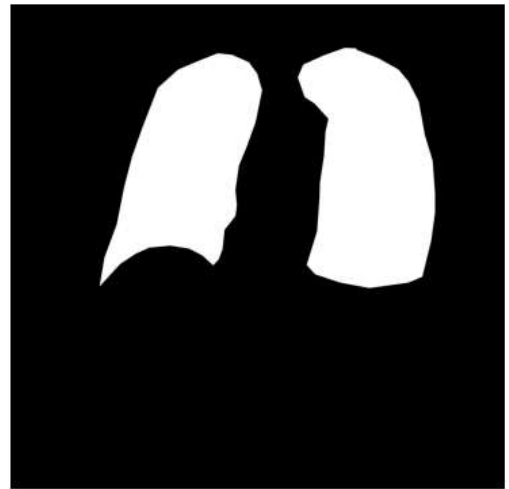


Image CHNCXR_0012_0.png



Mask

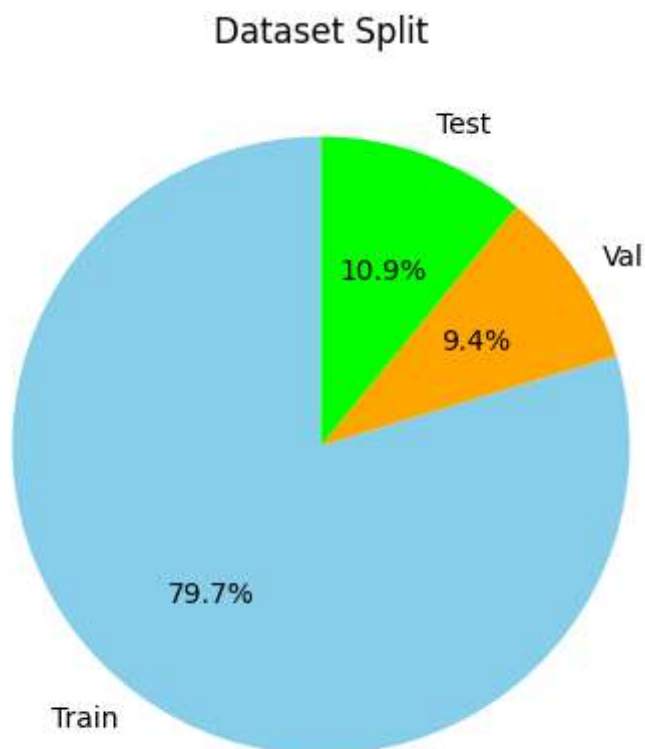


Image CHNCXR_0006_0.png



Mask





Task 0.1 : Preprocessing & Augmentation

- Implement preprocessing for images and masks.
- Ensure masks are not corrupted during transformations.

Image

- Decode as RGB (3 channels), resize to `IMG_SIZE` .
- Use ResNet preprocess.

Mask

- Decode as 1 channel, nearest resize (to preserve labels).
- Binarize.

Augmentations (mask-safe)

- Horizontal flip
- Light zoom/shift via pad + random crop
- Light brightness/contrast jitter (image only)

```
In [ ]: import importlib
import utilities as ut
importlib.reload(ut)
```

```

# Build datasets
train_ds = ut.make_dataset(train, augment_data=False) # no augmentation
val_ds   = ut.make_dataset(val, augment_data=False)
train_aug_ds = ut.make_dataset(train, augment_data=True)

# -----
# Check dataset sizes
# -----
print("Pairs available:")
print("  Train set:", len(train), "pairs")
print("  Val set  :", len(val), "pairs")
print("  Test set :", len(test), "pairs")

# Count actual batches and images
def count_batches_and_images(ds, name):
    n_batches = 0
    n_images = 0
    for imgs, msk in ds:
        n_batches += 1
        n_images += imgs.shape[0]
    print(f"{name}: {n_batches} batches, {n_images} images")

count_batches_and_images(train_ds, "Train dataset (no aug)")
count_batches_and_images(val_ds, "Validation dataset")
count_batches_and_images(train_aug_ds, "Train dataset (with aug)")

# -----
# Inspect shapes before augmentation
# -----
for imgs, msk in train_ds.take(1):
    print("Before augmentation - images:", imgs.shape, "masks:", msk.shape)

# Inspect shapes after augmentation
for imgs, msk in train_aug_ds.take(1):
    print("After augmentation - images:", imgs.shape, "masks:", msk.shape)

```

Pairs available:

Train set: 51 pairs

Val set : 6 pairs

Test set : 7 pairs

Train dataset (no aug): 13 batches, 51 images

Validation dataset: 2 batches, 6 images

Train dataset (with aug): 13 batches, 51 images

Before augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)

After augmentation - images: (4, 512, 512, 3) masks: (4, 512, 512, 1)

Part 1 - Model Implementation

Intersection over Union

The IoU score is a popular metric in both segmentation and object detection problems.

If you want to use the `plot_training_history` function in the `utilities.py` library remember to compile the model with the TP, TN, FP, FN metrics such that you can estimate the *Intersection-over-Union*.

However, it is voluntary to estimate IoU.(check the `utilities.py`)

See example below:

```
In [ ]: from tensorflow.keras.metrics import FalseNegatives, FalsePositives, TrueNegatives,
        from utilities import f1_score, plot_training_history

# Example model compile
model.compile(
    optimizer="Something else",
    loss="Something else",
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        f1_score,
        # Other metrics of your choice
    ]
)

training_history = model.fit(X_train, y_train, ...)
plot_training_history(training_history)
```

You have also been provided with a custom F1-score metric in the `utilities.py` library, which is specific for image segmentation.

This is mandatory to use when compiling the model.

Task 1.1: Model Implementation

Implement the classical U-net structure that you have learned about in the lectures. Feel free to experiment with the number of layers, loss-function, batch-normalization, etc.

Remember to compile with the F1-score metric.

```
In [ ]: # The encoder downsamples the input image to capture context and learn features.
        # This function builds one block of a U-Net encoder
        def encoder_block(inputs, num_filters):
            x = tf.keras.layers.Conv2D(num_filters, 3, padding='same')(inputs)
            x = tf.keras.layers.BatchNormalization()(x)
            x = tf.keras.layers.Activation('relu')(x)
            x = tf.keras.layers.Conv2D(num_filters, 3, padding='same')(x)
            x = tf.keras.layers.BatchNormalization()(x)
            x = tf.keras.layers.Activation('relu')(x)
            p = tf.keras.layers.MaxPool2D(pool_size=(2, 2), strides=2)(x)
            return p, x # return skip connection too (optional)

        # The decoder block makes the image bigger again and adds back details it learned e
```

```
# This function builds one block of a U-Net decoder
def decoder_block(inputs, skip_features, num_filters):
    x = tf.keras.layers.Conv2DTranspose(num_filters, (2, 2), strides=2, padding='sa
    x = tf.keras.layers.Concatenate()([x, skip_features])
    x = tf.keras.layers.Conv2D(num_filters, 3, padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = tf.keras.layers.Conv2D(num_filters, 3, padding='same')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    return x
```

```
In [ ]: input_shape = (512, 512, 3)
def unet():
    inputs = tf.keras.layers.Input(shape=input_shape)

    s1 = encoder_block(inputs, 64)
    s2 = encoder_block(s1[0], 128)
    s3 = encoder_block(s2[0], 256)
    s4 = encoder_block(s3[0], 512)

    b1 = tf.keras.layers.Conv2D(1024, 3, padding='same')(s4[0])
    b1 = tf.keras.layers.BatchNormalization()(b1)
    b1 = tf.keras.layers.Activation('relu')(b1)
    b1 = tf.keras.layers.Dropout(0.5)(b1)
    b1 = tf.keras.layers.Conv2D(1024, 3, padding='same')(b1)
    b1 = tf.keras.layers.BatchNormalization()(b1)
    b1 = tf.keras.layers.Activation('relu')(b1)
    b1 = tf.keras.layers.Dropout(0.5)(b1)

    d1 = decoder_block(b1, s4[1], 512)
    d2 = decoder_block(d1, s3[1], 256)
    d3 = decoder_block(d2, s2[1], 128)
    d4 = decoder_block(d3, s1[1], 64)

    outputs = tf.keras.layers.Conv2D(1, 1, activation='sigmoid', padding='same')(d4)

    model = tf.keras.models.Model(inputs, outputs)
    return model
```

```
In [ ]: from tensorflow.keras.metrics import FalseNegatives, FalsePositives, TrueNegatives,
from utilities import f1_score, plot_training_history, bce_dice_loss, iou_soft
# Compiling
model = unet()
model.compile(
    optimizer = "adam",
    loss = "binary_crossentropy",
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        f1_score,
        iou_soft,
```

```
]
)
```

Task 1.2: Train the model, and plot the training history

Feel free to use the `plot_training_history` function from the provided library `utilities.py`.

```
In [ ]: training_history = model.fit(
        train_aug_ds,
        validation_data = val_ds,
        epochs = 10,
    )
plot_training_history(training_history)
```

Epoch 1/10

13/13  **67s** 3s/step - f1_score: 0.5954 - false_negatives_9: 39863 2.9375 - false_positives_9: 1546225.2500 - iou_soft: 0.3210 - loss: 0.5709 - true_negatives_9: 4324303.0000 - true_positives_9: 1482811.2500 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 992.0000 - val_iou_soft: 1.5965e-11 - val_loss: 87.6752 - val_true_negatives_9: 1177483.0000 - val_true_positives_9: 0.0000e+00

Epoch 2/10

13/13  **23s** 1s/step - f1_score: 0.8079 - false_negatives_9: 48016 5.2188 - false_positives_9: 190548.8594 - iou_soft: 0.5458 - loss: 0.2553 - true_negatives_9: 5672494.5000 - true_positives_9: 1408764.0000 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 11395.0000 - val_iou_soft: 1.4997e-11 - val_loss: 5777.7852 - val_true_negatives_9: 1167080.0000 - val_true_positives_9: 0.0000e+00

Epoch 3/10

13/13  **40s** 1s/step - f1_score: 0.8455 - false_negatives_9: 38124 3.0000 - false_positives_9: 183383.6406 - iou_soft: 0.6043 - loss: 0.2040 - true_negatives_9: 5677239.5000 - true_positives_9: 1510106.2500 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.5529e-11 - val_loss: 4324.7393 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 4/10

13/13  **21s** 1s/step - f1_score: 0.8618 - false_negatives_9: 31323 2.7812 - false_positives_9: 156709.6406 - iou_soft: 0.6399 - loss: 0.1830 - true_negatives_9: 5694195.5000 - true_positives_9: 1587834.5000 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.5385e-11 - val_loss: 693.2032 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 5/10

13/13  **22s** 1s/step - f1_score: 0.8466 - false_negatives_9: 37406 3.6562 - false_positives_9: 195055.2812 - iou_soft: 0.6235 - loss: 0.2016 - true_negatives_9: 5629933.0000 - true_positives_9: 1552920.8750 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.5220e-11 - val_loss: 463.3514 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 6/10

13/13  **24s** 1s/step - f1_score: 0.8705 - false_negatives_9: 29245 4.4375 - false_positives_9: 146637.3594 - iou_soft: 0.6379 - loss: 0.1685 - true_negatives_9: 5747364.5000 - true_positives_9: 1565516.3750 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.6050e-11 - val_loss: 91.0588 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 7/10

13/13  **20s** 1s/step - f1_score: 0.8793 - false_negatives_9: 28017 2.3438 - false_positives_9: 155915.0781 - iou_soft: 0.6856 - loss: 0.1617 - true_negatives_9: 5654418.5000 - true_positives_9: 1661466.7500 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.8016e-04 - val_loss: 6.4305 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 8/10

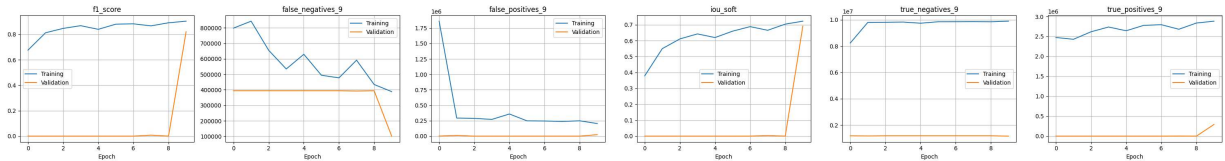
13/13  **22s** 1s/step - f1_score: 0.8610 - false_negatives_9: 36679 1.1562 - false_positives_9: 142013.1406 - iou_soft: 0.6678 - loss: 0.1823 - true_negatives_9: 5666909.0000 - true_positives_9: 1576259.1250 - val_f1_score: 0.0077 - val_false_negatives_9: 392227.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 0.0033 - val_loss: 6.8785 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 2162.0000

Epoch 9/10

13/13 ————— **21s** 1s/step - f1_score: 0.8929 - false_negatives_9: 251314.6406 - false_positives_9: 145063.5000 - iou_soft: 0.7033 - loss: 0.1414 - true_negatives_9: 5685886.5000 - true_positives_9: 1669708.0000 - val_f1_score: 0.0000e+00 - val_false_negatives_9: 394389.0000 - val_false_positives_9: 0.0000e+00 - val_iou_soft: 1.0789e-04 - val_loss: 4.5907 - val_true_negatives_9: 1178475.0000 - val_true_positives_9: 0.0000e+00

Epoch 10/10

13/13 ————— **21s** 997ms/step - f1_score: 0.9055 - false_negatives_9: 225358.6406 - false_positives_9: 117432.3594 - iou_soft: 0.7180 - loss: 0.1221 - true_negatives_9: 5763505.5000 - true_positives_9: 1645676.2500 - val_f1_score: 0.8210 - val_false_negatives_9: 101500.0000 - val_false_positives_9: 23645.0000 - val_iou_soft: 0.6914 - val_loss: 0.5247 - val_true_negatives_9: 1154830.0000 - val_true_positives_9: 292889.0000



Task 1.3: Visualize model predictions

Make a plot that illustrates the original image, the predicted mask, and the ground truth mask. You can use `visualize_predictions()` in `utilities.py`

```
In [ ]: from utilities import visualize_predictions
        visualize_predictions(model, val_ds)
```

1/1 ————— **2s** 2s/step

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



Task 2: Implementing U-Net with Transfer Learning

Implement a model with the U-net structure that you have learned about in the lectures, but now with a pre-trained backbone. There are many pre-trained back-bones to choose from. Pick freely from the selection here tf.keras.applications, or here [Keras model scores](#) (nicer table in the second link). Feel free to experiment with the number of layers, loss-function, batch-normalization, etc. Many of the backbones available are quite big, so you might find it quite time-consuming to train them on your personal computers.

For those with a dedicated graphics card (NVIDIA and AMD) Tensorflow or PyTorch (not syllabus)

And wants to experiment with their own compute resources (can be a lot of fun)

Tensorflow: <https://learn.microsoft.com/en-us/windows/ai/directml/gpu-tensorflow-plugin>

PyTorch: <https://learn.microsoft.com/en-us/windows/ai/directml/pytorch-windows>

Task 2.1: Transfer Learning Model Implementation

In this task, we will implement a U-Net model that uses the pre-trained weights of popular public networks (e.g., **VGG16** and **ResNet50**) as encoders. This is a common transfer

learning strategy for segmentation.

Example 1: U-Net with VGG16 backbone

```
from tensorflow.keras.applications import VGG16
```

```
from tensorflow.keras.applications import ResNet50
```

```
In [ ]: from tensorflow.keras.applications import VGG16
        from tensorflow.keras import models, layers

        def unet_with_vgg16():
            # Load VGG16 pretrained on ImageNet
            base_model = VGG16(weights='imagenet', include_top=False, input_shape=input_shape)

            # Encoder
            skips = [
                base_model.get_layer("block1_conv2").output,
                base_model.get_layer("block2_conv2").output,
                base_model.get_layer("block3_conv3").output,
                base_model.get_layer("block4_conv3").output,
            ]

            encoder_output = base_model.get_layer("block5_conv3").output # 16x16

            # Decoder
            x = encoder_output
            decoder_filters = [512, 256, 128, 64]

            for i, skip in enumerate(reversed(skips)):
                x = layers.Conv2DTranspose(decoder_filters[i], (3, 3), strides=(2, 2), padding='same')(x)
                x = layers.Concatenate()([x, skip])
                x = layers.Dropout(0.5)(x)

                x = layers.Conv2D(decoder_filters[i], (3, 3), padding='same')(x)
                x = layers.BatchNormalization()(x)
                x = layers.Activation('relu')(x)
                x = layers.Dropout(0.5)(x)

                x = layers.Conv2D(decoder_filters[i], (3, 3), padding='same')(x)
                x = layers.BatchNormalization()(x)
                x = layers.Activation('relu')(x)
                x = layers.Dropout(0.5)(x)

            outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(x)
            model = models.Model(inputs=base_model.input, outputs=outputs)
            return model
```

Task 2.2: Train the transfer learning model and plot the training history

Feel free to use the `plot_training_history` function from the provided library `utilities.py`

```
In [ ]: model2 = unet_with_vgg16()

model2.compile(
    optimizer = "adam",
    loss = "binary_crossentropy",
    metrics=[
        FalseNegatives(),
        FalsePositives(),
        TrueNegatives(),
        TruePositives(),
        f1_score,
        iou_soft,
    ]
)

history = model2.fit(
    train_aug_ds,
    validation_data = val_ds,
    epochs = 10,
    batch_size = 13,
)

plot_training_history(history)
```

Epoch 1/10

13/13 ————— **70s** 3s/step - f1_score: 0.4846 - false_negatives_14: 3278
17.0000 - false_positives_14: 2752582.7500 - iou_soft: 0.2767 - loss: 0.7735 - true_
negatives_14: 3105117.2500 - true_positives_14: 1566455.6250 - val_f1_score: 0.4735
- val_false_negatives_14: 199145.0000 - val_false_positives_14: 211267.0000 - val_iou_
soft: 0.2935 - val_loss: 0.7628 - val_true_negatives_14: 967208.0000 - val_true_posi
tives_14: 195244.0000

Epoch 2/10

13/13 ————— **22s** 1s/step - f1_score: 0.6401 - false_negatives_14: 3056
80.2188 - false_positives_14: 1350027.5000 - iou_soft: 0.3508 - loss: 0.4791 - true_
negatives_14: 4534030.5000 - true_positives_14: 1562234.5000 - val_f1_score: 0.7704
- val_false_negatives_14: 139026.0000 - val_false_positives_14: 10784.0000 - val_iou_
soft: 0.4886 - val_loss: 0.2787 - val_true_negatives_14: 1167691.0000 - val_true_posi
tives_14: 255363.0000

Epoch 3/10

13/13 ————— **22s** 1s/step - f1_score: 0.8042 - false_negatives_14: 3313
84.4375 - false_positives_14: 425316.0625 - iou_soft: 0.5082 - loss: 0.2877 - true_n
egatives_14: 5430632.0000 - true_positives_14: 1564640.0000 - val_f1_score: 0.6646 -
val_false_negatives_14: 189450.0000 - val_false_positives_14: 15285.0000 - val_iou_s
oft: 0.4393 - val_loss: 0.2910 - val_true_negatives_14: 1163190.0000 - val_true_posi
tives_14: 204939.0000

Epoch 4/10

13/13 ————— **23s** 1s/step - f1_score: 0.8362 - false_negatives_14: 3756
80.5000 - false_positives_14: 217208.1406 - iou_soft: 0.5774 - loss: 0.2278 - true_n
egatives_14: 5626335.5000 - true_positives_14: 1532748.2500 - val_f1_score: 0.3629 -
val_false_negatives_14: 308690.0000 - val_false_positives_14: 2937.0000 - val_iou_so
ft: 0.2863 - val_loss: 0.4822 - val_true_negatives_14: 1175538.0000 - val_true_posit
ives_14: 85699.0000

Epoch 5/10

13/13 ————— **22s** 1s/step - f1_score: 0.8930 - false_negatives_14: 2853
53.4375 - false_positives_14: 103321.0703 - iou_soft: 0.6437 - loss: 0.1626 - true_n
egatives_14: 5770532.0000 - true_positives_14: 1592766.0000 - val_f1_score: 0.7418 -
val_false_negatives_14: 4703.0000 - val_false_positives_14: 261247.0000 - val_iou_so
ft: 0.5489 - val_loss: 2.1982 - val_true_negatives_14: 917228.0000 - val_true_positi
ves_14: 389686.0000

Epoch 6/10

13/13 ————— **22s** 1s/step - f1_score: 0.8974 - false_negatives_14: 2335
26.2188 - false_positives_14: 131499.0000 - iou_soft: 0.6819 - loss: 0.1598 - true_n
egatives_14: 5727876.0000 - true_positives_14: 1659071.6250 - val_f1_score: 0.6157 -
val_false_negatives_14: 407.0000 - val_false_positives_14: 515732.0000 - val_iou_sof
t: 0.4343 - val_loss: 14.5291 - val_true_negatives_14: 662743.0000 - val_true_positi
ves_14: 393982.0000

Epoch 7/10

13/13 ————— **42s** 1s/step - f1_score: 0.8937 - false_negatives_14: 2491
86.5781 - false_positives_14: 104114.9297 - iou_soft: 0.6719 - loss: 0.1512 - true_n
egatives_14: 5791835.0000 - true_positives_14: 1606836.2500 - val_f1_score: 0.5385 -
val_false_negatives_14: 0.0000e+00 - val_false_positives_14: 703379.0000 - val_iou_s
oft: 0.3640 - val_loss: 13.3027 - val_true_negatives_14: 475096.0000 - val_true_posi
tives_14: 394389.0000

Epoch 8/10

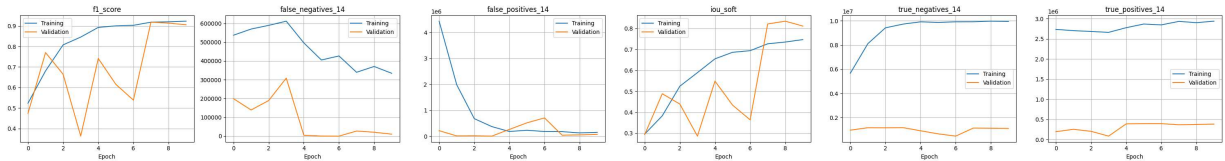
13/13 ————— **41s** 1s/step - f1_score: 0.9165 - false_negatives_14: 2190
68.6406 - false_positives_14: 97780.1406 - iou_soft: 0.7280 - loss: 0.1328 - true_ne
gatives_14: 5702217.5000 - true_positives_14: 1732906.3750 - val_f1_score: 0.9188 -
val_false_negatives_14: 27094.0000 - val_false_positives_14: 39267.0000 - val_iou_so
ft: 0.8215 - val_loss: 0.1865 - val_true_negatives_14: 1139208.0000 - val_true_posit
ives_14: 367295.0000

Epoch 9/10

13/13 ————— **22s** 1s/step - f1_score: 0.9139 - false_negatives_14: 235124.0000 - false_positives_14: 64187.3555 - iou_soft: 0.7242 - loss: 0.1292 - true_negatives_14: 5786402.5000 - true_positives_14: 1666258.6250 - val_f1_score: 0.9139 - val_false_negatives_14: 20673.0000 - val_false_positives_14: 51384.0000 - val_iou_soft: 0.8348 - val_loss: 0.2431 - val_true_negatives_14: 1127091.0000 - val_true_positives_14: 373716.0000

Epoch 10/10

13/13 ————— **22s** 1s/step - f1_score: 0.9218 - false_negatives_14: 205268.2188 - false_positives_14: 83887.4297 - iou_soft: 0.7474 - loss: 0.1202 - true_negatives_14: 5748380.5000 - true_positives_14: 1714436.5000 - val_f1_score: 0.9063 - val_false_negatives_14: 10448.0000 - val_false_positives_14: 69316.0000 - val_iou_soft: 0.8114 - val_loss: 0.2688 - val_true_negatives_14: 1109159.0000 - val_true_positives_14: 383941.0000



```
In [ ]: visualize_predictions(model2, val_ds)
```

WARNING:tensorflow:6 out of the last 7 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x78baffb774c0> triggered tf.function retracing. Tracing is expensive and the excessive number of tracings could be due to (1) creating @tf.function repeatedly in a loop, (2) passing tensors with different shapes, (3) passing Python objects instead of tensors. For (1), please define your @tf.function outside of the loop. For (2), @tf.function has reduce_retracing=True option that can avoid unnecessary retracing. For (3), please refer to https://www.tensorflow.org/guide/function#controlling_retracing and https://www.tensorflow.org/api_docs/python/tf/function for more details.

1/1 ————— **2s** 2s/step

WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-61.34..76.0305].



Part 2 - Discussion

Question 1: Which model architectures did you explore, and what type of hyperparameter optimization did you try?

Answer 1:

We tested a U-Net with a VGG16 backbone. The pretrained model gave better results compared to training from scratch.

For hyperparameters, we changed the batch size, and epochs. We tried batch sizes of 100, 10, and 13 depending on what the computer could handle. We also tried different sizes of epochs to see how much more the model could learn. We added dropout and batch normalization in the decoder to reduce overfitting. We also tried different loss functions like Binary Crossentropy, Dice loss, and we found out binary crossentropy was the better one.

Question 2: Which of the models did you choose to train as your final submission, and how long did it take to train?

Answer 2:

For the final model, we used the U-Net with a VGG16 encoder because it gave the most stable results and trained faster than the other versions. We used the pretrained VGG16

weights from ImageNet and kept most of the encoder layers frozen at first. Later in training, we unfroze the top layers to fine-tune them.

We trained the model on 512 by 512 images for 10 epochs with a batch size of 13. The optimizer we used was Adam. Training took around 5 minutes on a GPU

Question 3: What were the biggest challenges with this assignment?

Answer 3:

The biggest challenges were fixing input shape problems and dealing with memory limits. At first, we had an error because our dataset had 512 by 512 images, but the model was made for 256 by 256. It took some time to realize we needed to resize input_shape before training

Another challenge was choosing the right loss function. The model sometimes overfitted when using Dice loss, so we had to change it to Binary Crossentropy. We were also challenged with which activation function to use at the final layer. We tested with sigmoid and softmax. However, we found out that sigmoid was the one activation function for segmentation.